

Universidad de Málaga

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Proyecto IoT 2025

Simón Dice IoT: Juego de Memoria
Distribuido con ESP32 y MQTT



UNIVERSIDAD DE MÁLAGA

Autores: David Pérez Llobell, Salvador Patricio Lázaro Herrero y Manuel Gutiérrez Molina

Fecha: 29 de septiembre de 2026

Índice general

1. Introducción y Objetivos	3
1.1. Contexto del Proyecto	3
1.2. Objetivos	3
2. Diseño Hardware y Esquema de Conexión	4
2.1. Esquema de Conexión	4
2.2. Componentes Utilizados	5
3. Diseño Software del Sistema Distribuido	6
3.1. Arquitectura General del Sistema	6
3.1.1. Mapa de Comunicaciones MQTT	6
3.2. Subsistema 1: Controlador de Interfaz de Usuario (Teclado)	7
3.2.1. Descripción del Funcionamiento	7
3.2.2. Protocolo de Comunicación MQTT	8
3.2.3. Diagrama de Flujo del Firmware	8
3.2.4. Librerías Utilizadas	9
3.3. Subsistema 2: Interfaz de Salida y Feedback (LCD y Audio)	9
3.3.1. Descripción del Funcionamiento	9
3.3.2. Protocolo de Comunicación MQTT	10
3.3.3. Diagrama de Flujo del Firmware	11
3.3.4. Librerías Utilizadas	11
3.4. Subsistema 3: Controlador Lógico y Generador de Secuencias (LEDs)	11
3.4.1. Arquitectura del Firmware	12
3.4.2. Descripción del Funcionamiento	12
3.4.3. Protocolo de Comunicación MQTT	13
3.4.4. Diagrama de Flujo del Firmware	13
3.4.5. Librerías Utilizadas	14
3.5. Integración y Ciclo de Vida del Juego	14
3.6. Subsistema Transversal: Gestión de Actualizaciones FOTA	15
3.6.1. Arquitectura del Sistema de Actualización	15
3.6.2. Estrategia de Disparo (Triggers)	15
3.6.3. Flujo de Actualización	15
3.6.4. Diagrama de Lógica de Actualización	16
3.6.5. Librerías y Dependencias FOTA	16
4. Flujo en Nodered y base de datos	17
4.1. El flujo de Nodered	17
4.2. La base de datos	17
4.3. Telegram	19

5. Manual de Usuario	20
5.1. Configuración Inicial	20
5.1.1. Infraestructura Software	20
5.1.2. Configuración de la Red WiFi	21
5.1.3. Configuración de Credenciales MQTT	21
5.1.4. Puesta en Marcha del Sistema	21
5.2. Uso del Dashboard	21
5.3. Uso del Bot de Telegram y Guía de Usuario: Sistema Simón Dice IoT.	25
5.3.1. Inicio de la Comunicación (/start)	25
5.3.2. Registro del Jugador (/Nuevo_usuario [nombre])	25
5.3.3. Monitorización del Progreso (/Nivel_actual)	25
5.3.4. Consideraciones Importantes del Sistema	26
6. Resultados y Conclusiones	27
6.1. Resultados Obtenidos	27
6.2. Conclusiones y posibles mejoras	28

Capítulo 1

Introducción y Objetivos

1.1. Contexto del Proyecto

Se pretende desarrollar un juego de memoria a corto plazo con el objetivo de mejorar la capacidad de atención y retención de un público joven cuyos índices de dopamina están disparados del excesivo consumo de redes sociales y su capacidad de concentración cada vez es menor.

Para esto, se expone a continuación un proyecto de juego al más puro estilo de "Simón Dice", pero generado desde cero. Este proyecto hace uso de comunicación entre varios dispositivos ESP32 para poder monitorizar y controlar el uso del juego mediante MQTT, muestra y trata de datos en Node-Red y siguiendo el modelo FreeRTOS.

1.2. Objetivos

- Diseñar un sistema IoT distribuido con sensores y actuadores.
- Implementar control local y remoto mediante MQTT y Node-RED.
- Desarrollar una interfaz web (Dashboard) y un bot Telegram.
- Integrar una base de datos MongoDB para almacenamiento y consulta.

Capítulo 2

Diseño Hardware y Esquema de Conexión

2.1. Esquema de Conexión

Se presenta el esquemático del diseño final adjunto en la figura 2.1 que reproduce fielmente el montaje del proyecto.

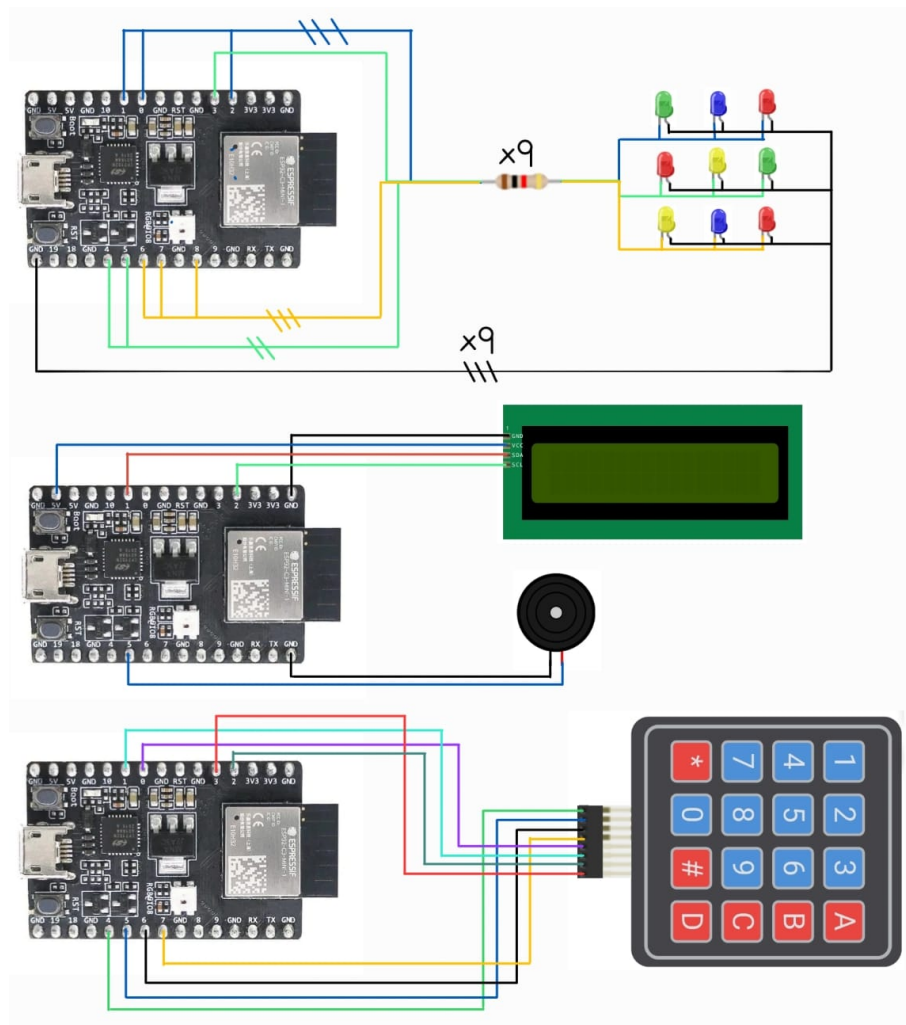


Figura 2.1: Esquema de conexión del dispositivo IoT.

2.2. Componentes Utilizados

A continuación se introduce un listado de sensores, actuadores y módulos empleados para el proyecto (ESP32-C3, LED RGB, etc.)

Componentes	Cantidad	Descripción
Diodo Led	9	Colores: Verde, Azul, Amarillo, Rojo
Resistencia (100 ohm)	9	Protección de los Leds
Cables	X	Interconexión de dispositivos
Display LCD I2C 5V PCF8574	1	Dimensiones 16x2
Zumbador pasivo 12085 42R	1	
Teclado matricial de membrana	1	Dimensiones 4x4
ESP32-DevC3	3	
Protoboard	4	

Cuadro 2.1: Tabla de componentes.

Capítulo 3

Diseño Software del Sistema Distribuido

3.1. Arquitectura General del Sistema

El sistema “Simón Dice IoT” se fundamenta en una arquitectura distribuida y modular, compuesta por tres nodos de procesamiento independientes (microcontroladores ESP32) que operan de forma cooperativa.

En lugar de centralizar toda la lógica en un único procesador, las funciones se han segregado en tres subsistemas especializados interconectados mediante una red WiFi y comunicados a través del protocolo MQTT. Esta estructura permite:

- Desacoplar la lectura de sensores (inputs) de la gestión de actuadores (outputs).
- Facilitar la escalabilidad del sistema.
- Mejorar la robustez mediante el aislamiento de tareas.

3.1.1. Mapa de Comunicaciones MQTT

La coordinación entre los tres nodos se realiza mediante un esquema de publicación/-suscripción. A continuación se define el diccionario de datos global del proyecto:

Topic	Tipo	Descripción
.../secuencia_actual	Global	Secuencia numérica generada que los nodos deben procesar.
.../resultado	Control	Feedback del nodo de entrada (OK/-FAIL).
.../estado_juego	Control	Sincronización de inicio/fin de partida.

Cuadro 3.1: Definición de Topics MQTT del sistema distribuido.

3.2. Subsistema 1: Controlador de Interfaz de Usuario (Teclado)

El firmware del dispositivo encargado del teclado se ha desarrollado sobre el framework de Arduino para el ESP32, utilizando el sistema operativo en tiempo real **FreeRTOS** para gestionar la concurrencia de procesos.

La arquitectura se simplifica en dos tareas principales que se ejecutan en paralelo para asegurar que la conectividad de red no bloquee la interacción con el usuario:

- **taskMQTTService:** Tarea encargada de la gestión de la conexión WiFi, el mantenimiento del enlace con el broker MQTT y la recepción de mensajes (callbacks).
- **JuegoTask:** Tarea dedicada exclusivamente a la lógica del juego, la lectura del teclado matricial y la validación inmediata de la secuencia introducida.

El sistema utiliza mecanismos de sincronización entre tareas, específicamente notificaciones directas (`xTaskNotifyGive` y `ulTaskNotifyTake`), para activar la lógica de juego solo cuando se recibe una nueva secuencia desde el servidor central.

3.2.1. Descripción del Funcionamiento

Inicialización y Conexión

Al arrancar, el sistema inicializa el puerto serie para depuración y crea las tareas de FreeRTOS. La tarea de comunicaciones intenta establecer conexión con la red WiFi configurada y posteriormente con el broker MQTT. Una vez conectado, el dispositivo se suscribe a los topics de control y publica su estado en línea.

Lógica de Recepción de Secuencia (Callback MQTT)

Cuando el sistema central genera una secuencia de juego, esta se recibe en el topic `secuencia_actual`. La función de callback realiza los siguientes pasos:

1. Recibe el payload en formato de cadena de texto (ej. "1,2,3,4").
2. Parsea la cadena separando los caracteres por comas y convierte los valores a enteros.
3. Almacena la secuencia en el array `receivedSequence`.
4. Desbloquea la tarea de juego mediante una notificación de tarea, indicando que hay una nueva partida lista.

Interacción con el Usuario (Keypad)

La tarea `JuegoTask` permanece en estado de bloqueo (`Blocked`) esperando la notificación. Al despertarse, entra en un bucle de lectura del teclado matricial 4x4.

El sistema espera a que el usuario pulse las teclas correspondientes a la secuencia recibida. Cada tecla pulsada se envía inmediatamente al topic `.../pulsacion` para permitir la monitorización en tiempo real.

Validación y Publicación de Resultados

La validación utiliza un modelo de “fallo rápido” (Fail-Fast). El firmware compara elemento a elemento el valor pulsado con el valor esperado en `receivedSequence` en el mismo instante de la pulsación.

- **Acierto:** Si el usuario completa toda la secuencia sin errores, se publica el mensaje “OK” en el topic de resultados.
- **Fallo Inmediato:** Si existe alguna discrepancia en cualquier pulsación, se publica “FAIL” inmediatamente y se termina la ronda sin esperar al final de la secuencia.

Tras la publicación, la tarea vuelve a dormirse esperando la siguiente ronda, liberando recursos de la CPU.

3.2.2. Protocolo de Comunicación MQTT

El dispositivo actúa como un nodo hoja en la arquitectura distribuida. Se han definido los siguientes topics para la comunicación:

Topic	Tipo	Descripción
.../secuencia_actual	Suscripción	Recibe la secuencia numérica a imitar (formato CSV).
.../pulsacion	Publicación	Envía en tiempo real cada tecla pulsada por el usuario.
.../resultado	Publicación	Envía el resultado de la validación (“OK” o “FAIL”).

Cuadro 3.2: Definición de Topics MQTT del nodo Teclado.

3.2.3. Diagrama de Flujo del Firmware

A continuación se muestra el flujo lógico simplificado de la tarea de juego:

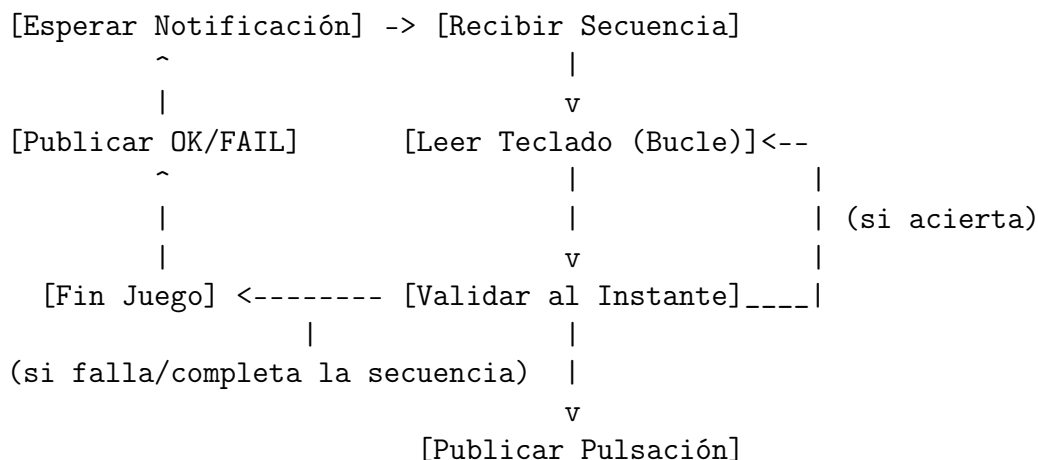


Figura 3.1: Diagrama de estados simplificado de la tarea de Juego.

3.2.4. Librerías Utilizadas

Para la implementación del código se han empleado las siguientes librerías estándar y de terceros:

- **Keypad.h**: Facilita el escaneo matricial de las filas y columnas del teclado, gestionando el “debouncing” (rebotes) de las teclas por software.
- **PubSubClient.h**: Cliente MQTT ligero para la publicación y suscripción de mensajes.
- **WiFi.h**: Gestión de la pila TCP/IP del ESP32.
- **FreeRTOS**: Primitivas del sistema operativo (incluidas en el core de ESP32) para la gestión de tareas (**TaskHandle_t**) y notificaciones.

3.3. Subsistema 2: Interfaz de Salida y Feedback (LCD y Audio)

El firmware de este subsistema gestiona la interacción con el usuario mediante señales visuales y sonoras. Al igual que el resto de nodos, está basado en el ESP32 y utiliza **FreeRTOS** para orquestrar la comunicación y el control de periféricos en tiempo real.

La arquitectura de software se estructura en dos tareas concurrentes que aseguran la reactividad del sistema:

- **taskMQTTService**: Gestiona la conectividad WiFi y la suscripción al broker MQTT. Sincroniza el inicio del resto de tareas mediante un semáforo binario (**semMqttReady**).
- **Controlador**: Tarea principal que implementa la máquina de estados del feedback. Controla el display LCD y genera melodías mediante el zumbador (buzzer) con una lógica de mensajes dinámica.

3.3.1. Descripción del Funcionamiento

Inicialización y Configuración de Hardware

En el arranque, el sistema configura explícitamente el bus I2C en los pines **GPIO 1 (SDA)** y **GPIO 2 (SCL)** mediante la instrucción **Wire.begin(1, 2)**. Posteriormente, inicializa la pantalla LCD y configura los pines del zumbador.

Se utiliza un semáforo binario (**semMqttReady**) que bloquea la lógica de control hasta que la conexión MQTT es estable, asegurando que no se intenten publicar estados sin conexión.

Gestión Dinámica de Información en Pantalla (LCD)

La pantalla LCD actúa como el panel de información principal. En esta versión del firmware, se ha dotado al sistema de una "personalidad" mediante mensajes rotativos para evitar la monotonía:

- **Modo Reposo** (**comparacion == 3**): Muestra el título del juego: "SIMON DICE: JUEGO DE MEMORIA".

- **Modo Juego:** Muestra en tiempo real el nivel actual o el usuario activo mediante eventos MQTT directos.
- **Feedback de Resultado (Rotativo):** Se utilizan contadores cíclicos (`frases_good` y `frases_bad`) para rotar entre 4 mensajes diferentes según el desempeño:
 - **Aciertos:** Mensajes de refuerzo como “Sigue así !!!”, “Eres un makina”, “Estas en racha” o “Que memoria...”.
 - **Fallos:** Mensajes de error como “Ahhh fallaste”, “Eres lamentable”, “No vales para na” o “Memoria de pez”.

Feedback Auditivo y Señalización

La función `alarma()` genera patrones sonoros sincronizados con el resultado visual:

- **Acierto:** Reproduce una melodía ascendente y alegre.
- **Fallo:** Reproduce una secuencia de tonos graves y descendentes.

Simultáneamente, el sistema publica un “1” (acierto) o “0” (fallo) en el topic `.../led` para activar una señalización luminosa externa.

Máquina de Estados

El flujo lógico de la tarea `Controlador` se gobierna mediante la variable global `comparacion`:

- **Estado 1 (OK):** Selecciona la siguiente frase de éxito, reproduce sonido, activa LED externo y pasa a espera.
- **Estado 0 (FAIL):** Selecciona la siguiente frase de fallo, reproduce sonido de error, apaga LED externo y pasa a espera.
- **Estado 3 (Idle):** Gestiona la pantalla de título.

3.3.2. Protocolo de Comunicación MQTT

Este nodo actúa principalmente como suscriptor para recibir órdenes de visualización.

Topic	Tipo	Descripción
<code>.../comparacion</code>	Suscripción	Recibe “OK” o “FAIL”. Dispara la lógica de mensajes rotativos y sonidos.
<code>.../nivel_actual</code>	Suscripción	Recibe el número de nivel para mostrarlo en pantalla.
<code>.../muestra_usuario</code>	Suscripción	Recibe el nombre del jugador activo.
<code>.../led</code>	Publicación	Envía “1” o “0” para control de iluminación externa.

Cuadro 3.3: Definición de Topics MQTT del nodo Feedback.

3.3.3. Diagrama de Flujo del Firmware

A continuación se muestra el flujo lógico de la tarea **Controlador**:

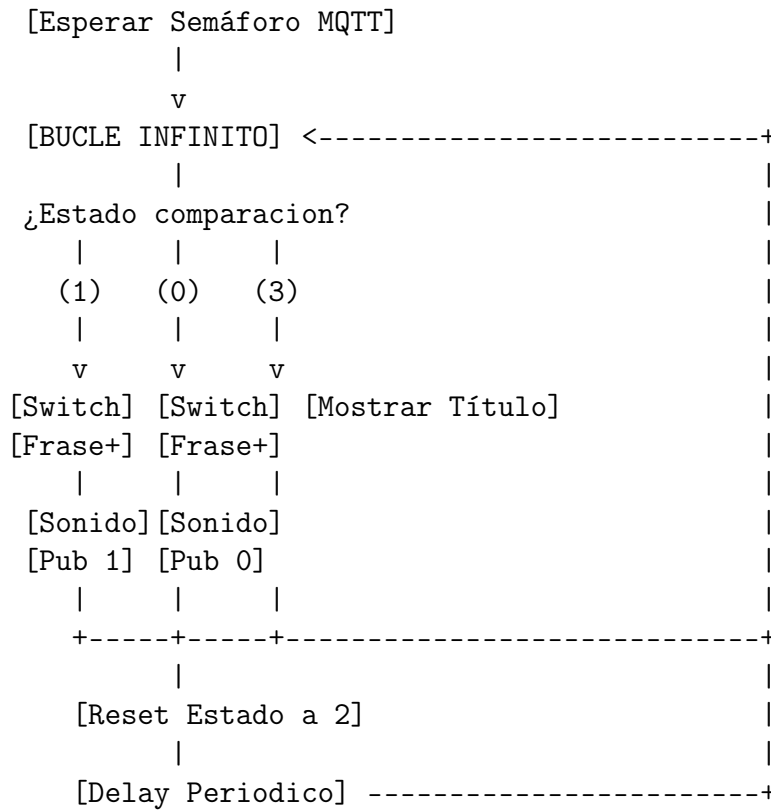


Figura 3.2: Diagrama de estados simplificado de la tarea **Controlador** con frases dinámicas.

3.3.4. Librerías Utilizadas

Para la implementación se han añadido librerías específicas para el control del hardware:

- `LiquidCrystal_I2C.h`: Control del LCD.
- `Wire.h`: Comunicación I2C (configurada en pines 1 y 2).
- `PubSubClient.h`: Cliente MQTT.
- `WiFi.h`: Gestión de red.
- `FreeRTOS`: Gestión de tareas y sincronización.

3.4. Subsistema 3: Controlador Lógico y Generador de Secuencias (LEDs)

Este nodo actúa como el "cerebro" central del juego. Es responsable de la generación pseudoaleatoria de las secuencias que el usuario debe memorizar, así como de la gestión del nivel de dificultad y la visualización directa del patrón mediante una matriz de LEDs.

3.4.1. Arquitectura del Firmware

Al igual que los otros nodos, se basa en el ESP32 corriendo sobre FreeRTOS. Su estructura se centra en la reactividad ante los resultados de la partida para ajustar la dificultad dinámicamente.

Se compone principalmente de dos tareas de ejecución:

- **taskMQTTService:** Mantiene la conexión permanente con el broker. A diferencia de otros nodos, aquí el callback MQTT es crítico porque dispara la lógica de generación de nuevas secuencias.
- **OTATriggerTask:** Tarea auxiliar de mantenimiento encargada de la monitorización del sistema.

3.4.2. Descripción del Funcionamiento

Generación de Secuencias (`calculaCombinacion`)

La función principal del juego es `calculaCombinacion(int nivel)`. Su lógica es incremental para aumentar la dificultad progresivamente:

1. **Reinicio:** Si el nivel es 1 (inicio de partida), se limpia el vector de secuencia (`std::vector<int>secuencia`).
2. **Generación:** Se genera un nuevo número aleatorio entre 0 y 8 (correspondiente a los LEDs físicos).
3. **Acumulación:** Este nuevo paso se añade al final del vector existente.
4. **Serialización:** Se convierte el vector numérico a una cadena de texto separada por comas (ej: "3,0,7") para su transmisión.

Visualización Física

Antes de enviar la secuencia a la red, el propio nodo la reproduce físicamente para que el usuario pueda memorizarla. Utiliza un bucle que recorre el vector `secuencia`, encendiendo el pin GPIO correspondiente durante 500ms y apagándolo otros 500ms. Esto proporciona el estímulo visual que el jugador debe replicar posteriormente en el teclado.

Gestión de Niveles y Lógica de Partida

El flujo del juego se controla dentro de la función de callback MQTT (`procesa_mensaje`), reaccionando al topic de resultados:

- **Mensaje "OK" (Acierto):**
 1. Incrementa el contador de `nivel`.
 2. Llama inmediatamente a `calculaCombinacion` para añadir un paso extra y mostrar la nueva secuencia más larga.
- **Mensaje "FAIL" (Error):**

1. Publica el nivel alcanzado en el topic de almacenamiento (`.../almacena_nivel`) para registro de puntuaciones.
2. Reinicia el contador de `nivel` a 1.
3. Genera una nueva secuencia simple de un solo paso para comenzar una nueva partida.

3.4.3. Protocolo de Comunicación MQTT

Este nodo actúa como el orquestador, publicando la información que consumen tanto el teclado (para validación) como la pantalla LCD (para información).

Topic	Tipo	Descripción
<code>.../comparacion</code>	Suscripción	Recibe “OK” o “FAIL” desde el teclado para decidir si avanzar nivel o reiniciar.
<code>.../secuencia_actual</code>	Publicación	Envía la secuencia completa (CSV) que el teclado debe validar.
<code>.../nivel_actual</code>	Publicación	Informa del nivel de dificultad actual para que el LCD lo muestre.
<code>.../almacena_nivel</code>	Publicación	Al perder, envía la puntuación final para su registro en base de datos.

Cuadro 3.4: Definición de Topics MQTT del nodo Generador.

3.4.4. Diagrama de Flujo del Firmware

A continuación se muestra el ciclo de vida de la lógica de juego en este nodo:

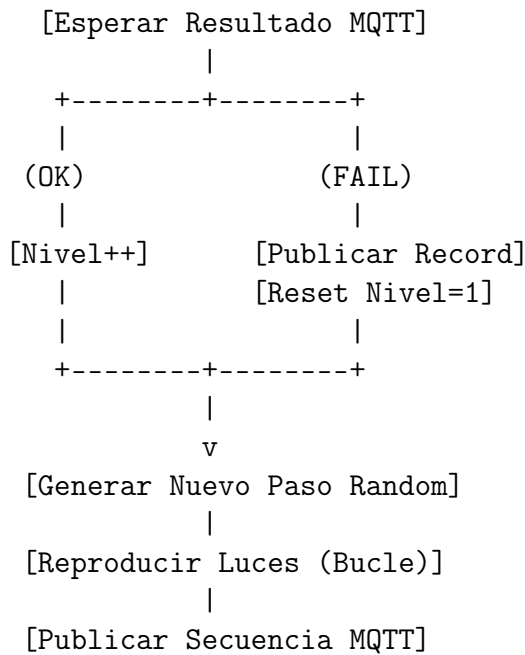


Figura 3.3: Diagrama de estados de la generación de secuencias.

3.4.5. Librerías Utilizadas

Además de las librerías de comunicación estándar, se utiliza la librería estándar de C++ para gestión dinámica de memoria:

- `<vector>`: Contenedor dinámico de la STL de C++ utilizado para almacenar la secuencia de luces, permitiendo que crezca indefinidamente según avanza el nivel.
- `PubSubClient.h`: Cliente MQTT.
- `WiFi.h`: Gestión de red.
- `FreeRTOS`: Gestión de tareas en segundo plano.

3.5. Integración y Ciclo de Vida del Juego

El funcionamiento conjunto de los tres subsistemas define el ciclo de juego completo. La interacción sigue el siguiente diagrama de secuencia lógico:

1. El **Subsistema 3** inicia la ronda generando una secuencia aleatoria y publicándola en MQTT.
2. El **Subsistema 2** recibe la secuencia y ofrece el feedback visual/sonoro al usuario.
3. El **Subsistema 1** (Teclado) habilita la lectura de input.
4. Tras la entrada del usuario, el **Subsistema 1** valida y comunica el resultado.
5. Basado en el resultado, el **Subsistema 3** decide si incrementar la dificultad o reiniciar la partida.

3.6. Subsistema Transversal: Gestión de Actualizaciones FOTA

Dada la arquitectura distribuida del sistema y la dificultad de acceder físicamente a cada nodo para su mantenimiento, se ha implementado un mecanismo de actualización de firmware por aire (FOTA - *Firmware Over-The-Air*). Este módulo es común a todos los dispositivos ESP32 de la red y permite el despliegue de nuevas versiones de software de manera centralizada mediante el protocolo HTTP.

3.6.1. Arquitectura del Sistema de Actualización

El sistema se basa en un modelo cliente-servidor donde los microcontroladores actúan como clientes HTTP que consultan periódicamente a un servidor central de despliegue.

- **Servidor de Actualización:** Un servidor HTTP aloja los archivos binarios compilados (.bin). Este puede ser local o el disponible en `iot.ac.uma.es`. En nuestro caso hemos optado por usar el local.
- **Gestión de Versiones:** El firmware incluye una macro de preprocesador (`HTTP_OTA_VERSION`) que genera dinámicamente una cadena de versión basada en el nombre del archivo fuente y su fecha de compilación. Esta cadena se envía en las cabeceras HTTP para que el servidor determine si es necesaria una actualización.

3.6.2. Estrategia de Disparo (Triggers)

Para garantizar tanto la autonomía del sistema como la capacidad de intervención manual, se ha diseñado una tarea específica de monitoreo (`OTATriggerTask`) que gestiona tres eventos de disparo distintos:

1. **Verificación al Inicio (Boot):** Cada vez que el dispositivo se reinicia (ya sea por un corte de energía o un reset software), se lanza inmediatamente una petición de actualización antes de iniciar las tareas críticas del sistema.
2. **Verificación Periódica (Timer):** Se utiliza un temporizador por software que comprueba la existencia de nuevas versiones cada 10 minutos (`OTA_INTERVAL_MS = 600000`). Esto asegura que todos los nodos converjan a la última versión en un tiempo razonable sin saturar la red.
3. **Disparo Manual (Hardware):** Se ha habilitado el botón `BOOT` (GPIO 9 en ESP32-C3) como interruptor de mantenimiento. Al mantenerlo pulsado, el técnico fuerza una comprobación inmediata de actualización, útil para pruebas de campo.

3.6.3. Flujo de Actualización

El proceso de actualización se ejecuta en una tarea dedicada (`OTA_Task`) con alta prioridad y stack suficiente (8192 bytes) para manejar la conexión HTTP y las operaciones de escritura en la memoria flash. El flujo es el siguiente:

1. **Conexión:** Se establece una conexión TCP con el servidor OTA.

2. **Handshake:** El cliente envía su cabecera de versión actual. Si el servidor responde con HTTP_UPDATE_NO_UPDATES, el proceso termina.
3. **Descarga y Flasheo:** Si hay una actualización, la librería HTTPUpdate gestiona la descarga del binario y su escritura secuencial en la partición OTA de la memoria flash.
4. **Callbacks de Estado:** Durante el proceso, el sistema ejecuta funciones de callback para notificar el inicio, el progreso (porcentaje de descarga) y la finalización o error a través del puerto serie.
5. **Reinicio:** Tras una escritura exitosa (HTTP_UPDATE_OK), el ESP32 se reinicia automáticamente para cargar el nuevo firmware.

3.6.4. Diagrama de Lógica de Actualización

A continuación se presenta el esquema de decisión implementado en la tarea OTATriggerTask:

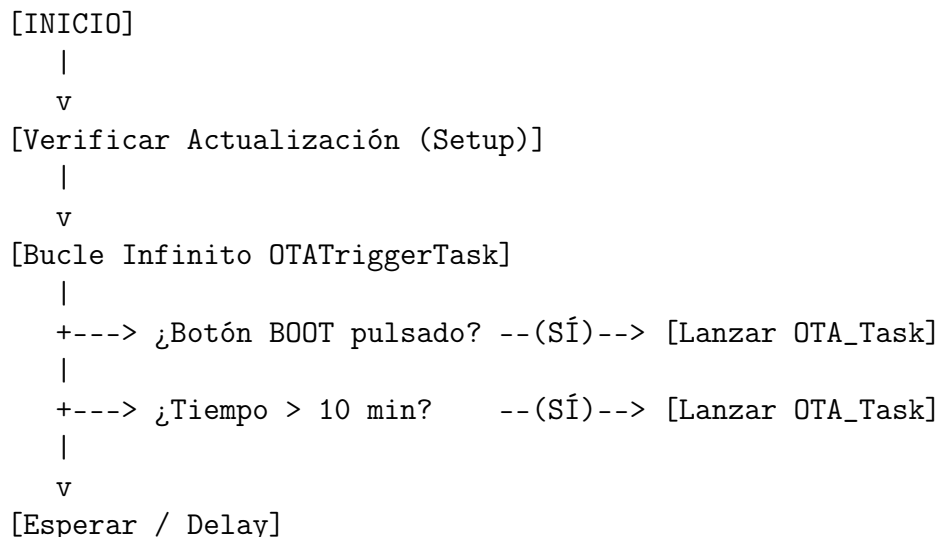


Figura 3.4: Diagrama de flujo de los disparadores de actualización FOTA.

3.6.5. Librerías y Dependencias FOTA

Para la implementación de este módulo común se han utilizado las siguientes librerías del core de ESP32:

- `HTTPClient.h`: Para la gestión base de las peticiones web.
- `HTTPUpdate.h`: Librería de alto nivel que automatiza la negociación de versiones y la escritura en la memoria particionada.

Capítulo 4

Flujo en Nodered y base de datos

4.1. El flujo de Nodered

Las interfaces y la base de datos están implementadas con nodered haciendo uso de las capacidades que ofrece el software. A continuación se va a introducir el esquema de diseño que se ha empleado, es decir, el flujo.

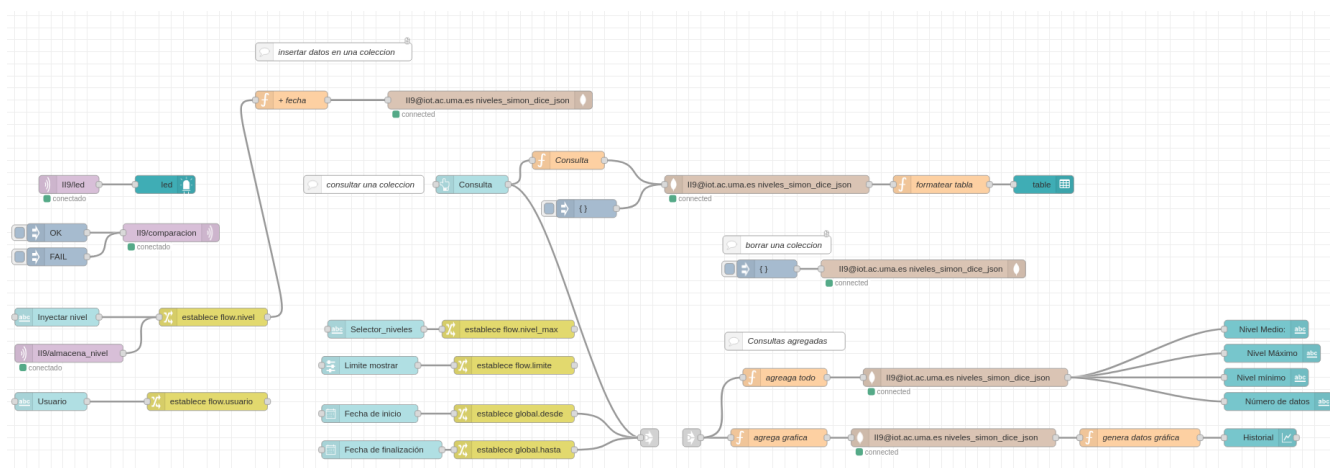


Figura 4.1: Flujo de nodered.

4.2. La base de datos

El primer bloque que encontramos en el flujo es el que implementa todas las funcionalidades que ofrece el dashboard las cuales se apoyan en la base de datos la que hace uso de un servidor MongoDB asignado a nuestro grupo de trabajo.

Para cada nueva entrada, se formatea previamente en un Json mediante una función que le añade la fecha y el usuario que ha alcanzado el nivel indicado y finalmente se almacena en la base de datos.

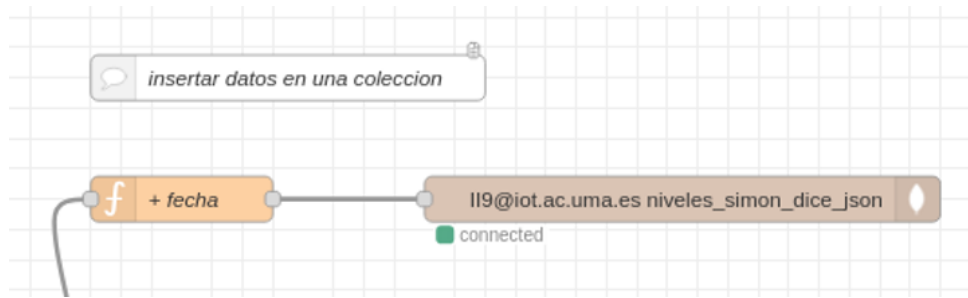


Figura 4.2: Ingreso en la base de datos.

También existe la funcionalidad que permite el borrado de la base de datos. Solo se puede aplicar desde el editor de nodered, a nivel de administrador para evitar que el jugador, sin darse cuenta, elimine de forma accidental todos los registros.

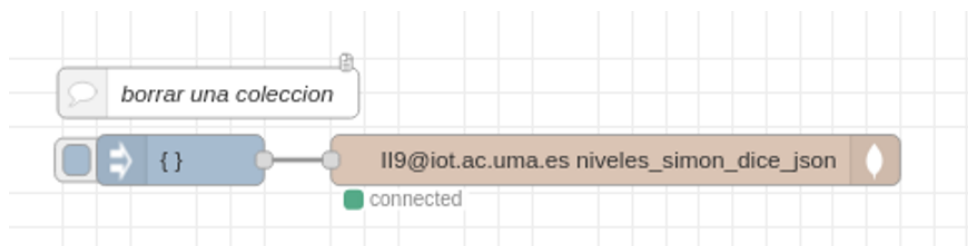


Figura 4.3: Borrado de la base de datos.

El jugador puede insertar parámetros para la consulta de la base de datos mediante selectores de fecha,deslizaderas y líneas de texto para introducir valores.

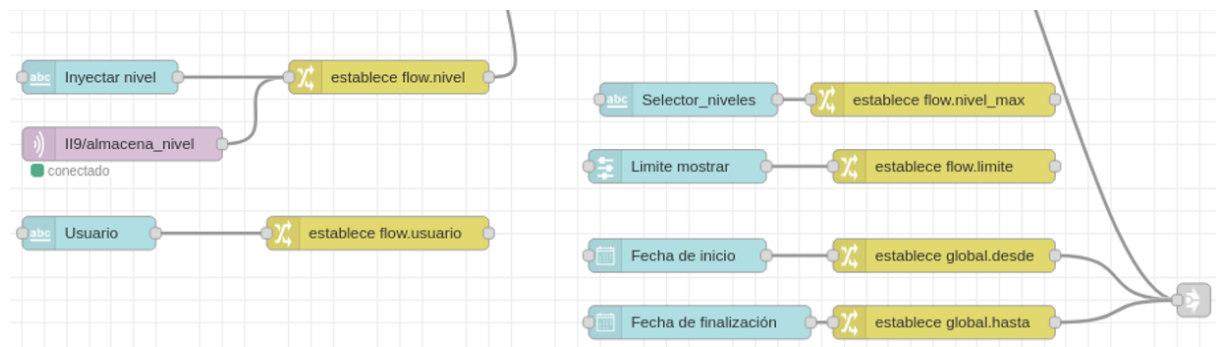


Figura 4.4: Ingreso de parámetros de búsqueda.

La información almacenada en la base de MongoDB se visualiza mediante dos caminos diferentes, por una parte tenemos una tabla que muestra todos los registros siguiendo un criterio de selección que se puede configurar desde el propio dashboard de forma dinámica.

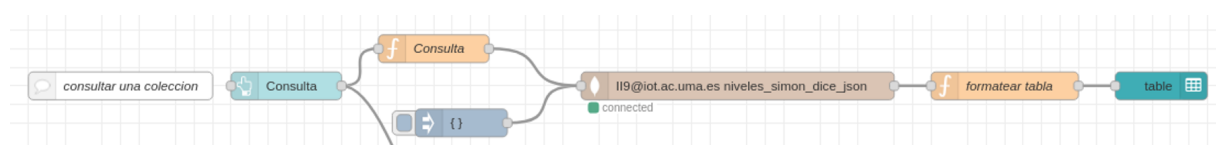


Figura 4.5: Tabla y consulta de la base de datos.

Por otra parte se cuenta con 2 agregaciones que se encargan de calcular estadísticas del usuario y de realizar un gráfico de los mismos siguiendo un formato especificado por la función que lo alimenta.

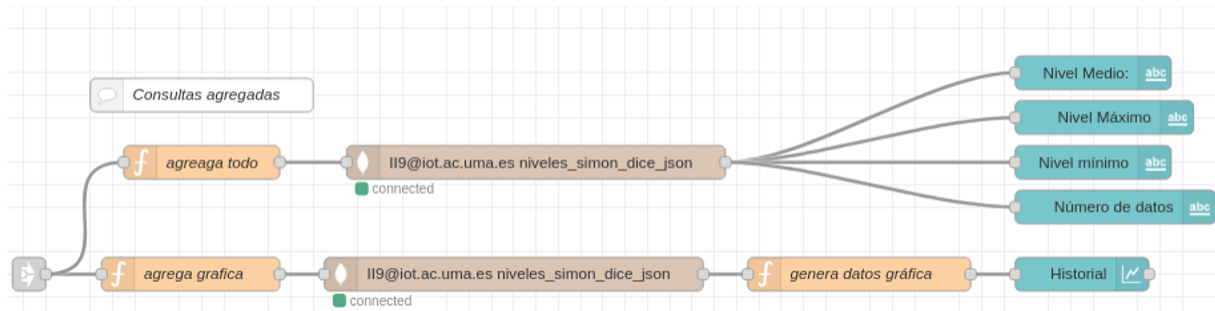


Figura 4.6: Agregación e historial gráfico.

4.3. Telegram

El segundo bloque contenido en el flujo engloba todo lo relacionado con la interfaz de telegram.

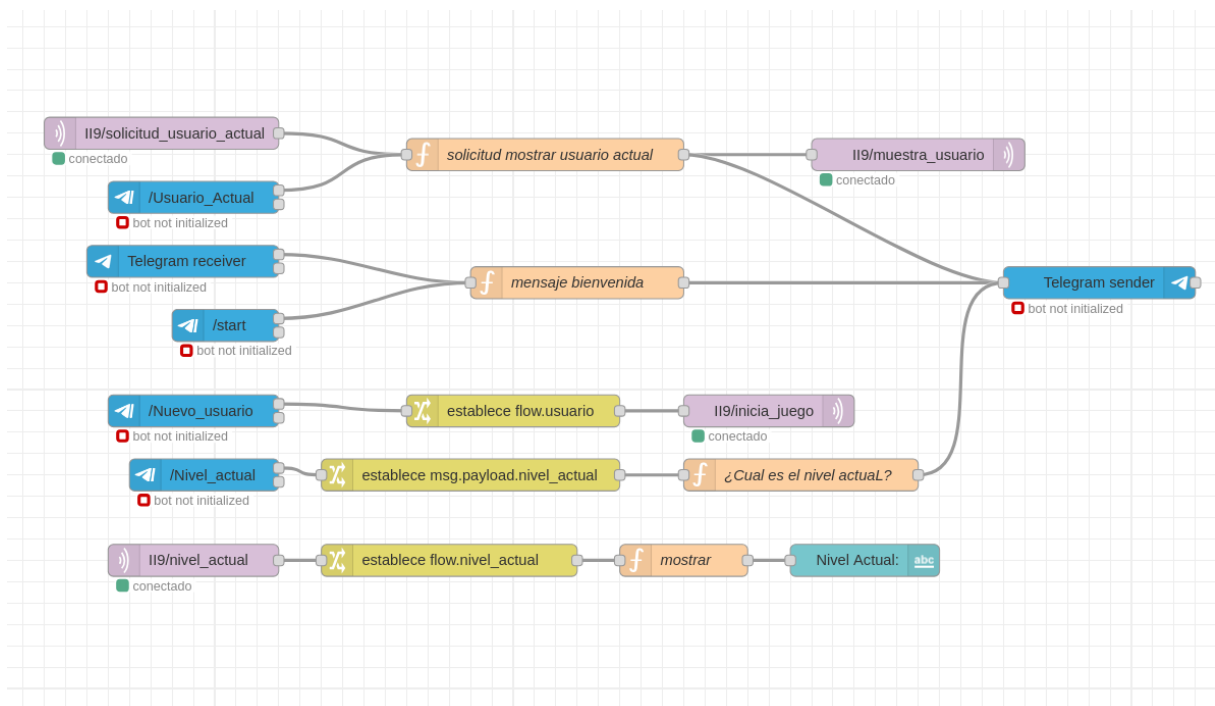


Figura 4.7: Implementación de telegram.

Los diferentes nodos se encargan de activar las funciones del flujo y de establecer los nuevos valores de las variables de flujo empleadas por el resto de funciones. También publican por los topics del servidor MQTT para mandar a las placas información útil para su funcionamiento.

Capítulo 5

Manual de Usuario

5.1. Configuración Inicial

El sistema ha sido diseñado para que su puesta en marcha sea lo más sencilla posible, minimizando la necesidad de configuración manual por parte del usuario final. Una vez desplegada la infraestructura software y cargado el firmware en los dispositivos, el sistema queda completamente operativo.

5.1.1. Infraestructura Software

Para el correcto funcionamiento del sistema es suficiente con disponer de un entorno Node-RED ejecutándose sobre Docker Desktop, donde se alojan todos los servicios necesarios para la operación del proyecto. En este entorno se integran:

- **El broker MQTT, encargado de la comunicación entre los distintos nodos ESP32.**
- **El dashboard web, que permite la visualización del estado del juego y la consulta de resultados.**
- **La base de datos MongoDB, utilizada para el almacenamiento persistente de las partidas y estadísticas.**
- **La integración con Telegram, que actúa como interfaz remota adicional.**

Una vez levantados los contenedores Docker correspondientes, desplegado el flujo de Node-RED y encendido el bot de telegram, todos los servicios quedan disponibles sin necesidad de configuraciones adicionales.

Instalación del Firmware en los Dispositivos ESP32

Cada uno de los nodos del sistema (teclado, generador de secuencias y módulo de feedback) dispone de su propio firmware específico. La instalación se realiza mediante el entorno de desarrollo Arduino IDE o plataforma equivalente:

1. **Conectar cada ESP32 al ordenador mediante USB.**
2. **Seleccionar el modelo de placa correspondiente (ESP32-C3) y el puerto serie adecuado.**
3. **Compilar y cargar el firmware correspondiente a cada subsistema.**

4. Verificar mediante el monitor serie que el dispositivo se conecta correctamente a la red WiFi y al broker MQTT.

Una vez cargado el firmware, no es necesario volver a conectar físicamente los dispositivos al ordenador, ya que el sistema dispone de actualización remota (FOTA).

5.1.2. Configuración de la Red WiFi

Los dispositivos ESP32 están configurados para conectarse automáticamente a la red WiFi del laboratorio de prácticas, que proporciona acceso a la red local y a los servicios alojados en Docker.

Las credenciales de la red WiFi se encuentran definidas en el firmware, por lo que el usuario no necesita introducir ningún dato adicional durante el arranque del sistema. No obstante, el diseño permite modificar fácilmente estas credenciales para adaptarse a otros entornos de red si fuese necesario.

5.1.3. Configuración de Credenciales MQTT

La conexión al broker MQTT se realiza de forma automática al iniciar los dispositivos. Los parámetros necesarios (dirección del broker, puerto y topics utilizados) están preconfigurados en el código de los ESP32 y en el flujo de Node-RED.

Dado que el broker MQTT se encuentra alojado dentro del entorno Docker, no es necesario realizar configuraciones adicionales ni gestionar direcciones IP externas. Todos los nodos publican y se suscriben a los topics definidos siguiendo una estructura común, garantizando la correcta sincronización del sistema.

5.1.4. Puesta en Marcha del Sistema

Una vez cumplidos los pasos anteriores, el sistema queda completamente operativo. Basta con:

- Tener Node-RED y los servicios asociados en ejecución mediante Docker.
- Alimentar los dispositivos ESP32.
- Acceder al dashboard web para monitorizar y controlar el juego.

A partir de este punto, el sistema inicia automáticamente el ciclo de juego, permitiendo la interacción con el usuario y el registro de resultados sin intervención adicional.

5.2. Uso del Dashboard

El sistema cuenta con 2 formas de interactuar con el usuario, la primera es el dashboard de nodered.

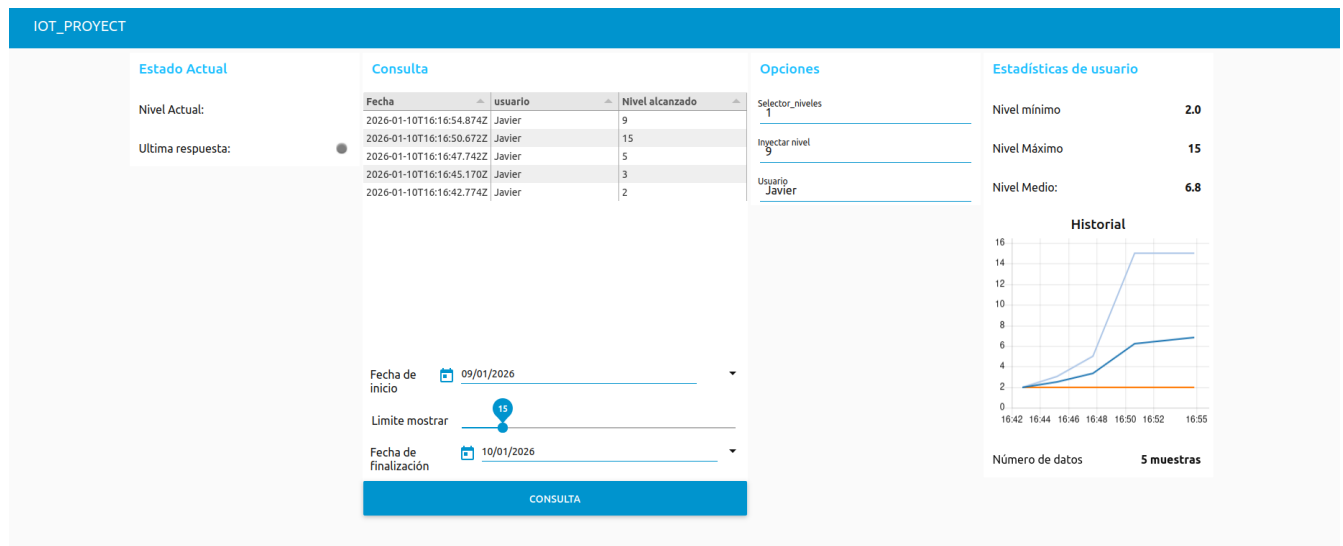


Figura 5.1: Interfaz gráfica en Node-RED Dashboard.

La interfáz de nodered se organiza en cuatro bloques interrelacionados entres sí. A la izquierda se presenta información relevante en tiempo real del sisitema que se va proporcionando por las placas.

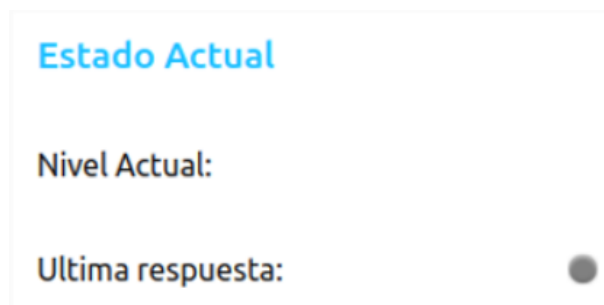


Figura 5.2: Bloque de estado.

En el apartado de consulta, se encuentra la tabla con todos los registros de las partidas jugadas por los usuarios. Cuenta con unos modificadores para seleccionar el periodo de tiempo en que se desean los datos y el número de registros.

Consulta

Fecha	usuario	Nivel alcanzado
2026-01-10T16:16:54.874Z	Javier	9
2026-01-10T16:16:50.672Z	Javier	15
2026-01-10T16:16:47.742Z	Javier	5
2026-01-10T16:16:45.170Z	Javier	3
2026-01-10T16:16:42.774Z	Javier	2

Fecha de inicio  09/01/2026

Limite mostrar  15

Fecha de finalización  10/01/2026

CONSULTA

Figura 5.3: Bloque de consulta.

A su derecha, se encuentra el bloque de opciones que se encarga de seleccionar el nombre del jugador del que se van a visualizar sus estadísticas e información. También se cuenta con una línea que permite seleccionar el nivel mínimo desde el que se desea visualizar registros.

Adicionalmente, a modo de prueba, se pueden asignar a los usuarios nuevos registros. Para ello hay que introducir el usuario deseado y a continuación introducir el nivel que se desee en la línea de inyectar nivel.

Opciones

Selector_niveles
1

Injectar nivel
9

Usuario
Javier

Figura 5.4: Bloque de opciones.

Finalmente, a la derecha de la pantalla, se encuentran las estadísticas de usuario, que comprende los niveles máximo, mínimo a los que ha llegado y la media de nivel del jugador. Debajo se encuentra un gráfico para visualizar de forma más descriptiva la trayectoria que ha seguido el jugador durante el periodo de tiempo seleccionado.

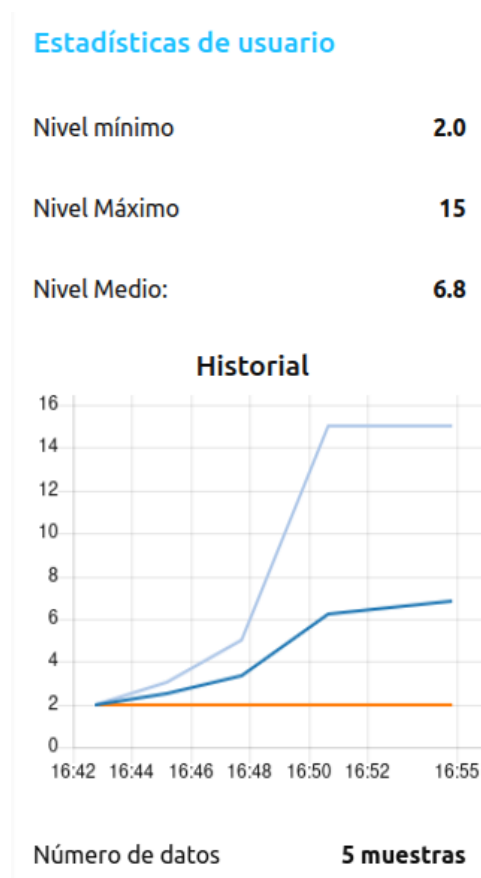


Figura 5.5: Bloque de estadísticas de usuario.

5.3. Uso del Bot de Telegram y Guía de Usuario: Sistema Simón Dice IoT.

El bot de Telegram actúa como herramienta de control remoto y supervisión del sistema. Permite al usuario registrarse para jugar y monitorizar el progreso de la partida en tiempo real desde cualquier lugar con conexión a Internet.

5.3.1. Inicio de la Comunicación (/start)

Para comenzar, el usuario debe abrir el chat con el bot y enviar el comando `/start` o con cualquier cadena de texto que no corresponda con un comando válido. El sistema reconocerá su presencia y responderá con un mensaje de bienvenida que actúa como menú inicial de instrucciones.

- **Lo que verás en pantalla:** mensaje de bienvenida con las instrucciones básicas de uso del bot.

5.3.2. Registro del Jugador (/Nuevo_usuario [nombre])

Antes de interactuar con la consola física, es necesario identificar al jugador. Para ello, se debe enviar el comando `/Nuevo_usuario` seguido del nombre del jugador (por ejemplo, `/Nuevo_usuario David`).

- **¿Qué hace el sistema?** En este punto, el “cerebro” del juego captura el nombre del usuario y lo almacena en su base de datos (MongoDB). Aunque el bot no envía una confirmación directa en el chat, el nombre se transmite correctamente a la consola física para personalizar la partida.
- **Inicio del juego:** Una vez registrado el nombre, la consola de LEDs comienza automáticamente la generación de la primera secuencia. A partir de ese momento, el jugador debe memorizarla para continuar avanzando.

5.3.3. Monitorización del Progreso (/Nivel_actual)

Para conocer el rendimiento del jugador o supervisar la partida de otro usuario, se puede emplear el comando `/Nivel_actual`. El bot consulta al controlador del juego y responde con un mensaje personalizado que refleja el estado actual de la memoria del jugador.

- **Lo que verás en pantalla:** mensaje informativo indicando el nivel alcanzado por el jugador.
- **Nota sobre el funcionamiento:** El mensaje es dinámico. Si el jugador falla una secuencia, el nivel se reinicia automáticamente a 1 y este cambio se refleja en la siguiente consulta realizada al bot.

5.3.4. Consideraciones Importantes del Sistema

- **Sin confirmaciones intermedias:** El sistema está diseñado para ser ágil. Al registrar un usuario, no se envía un mensaje explícito de confirmación (*OK*); el cambio se refleja directamente en la consola física (pantalla LCD y LEDs).
- **Actualización de errores:** Cuando el jugador falla una secuencia introducida mediante el teclado físico, el sistema detecta el error de forma inmediata, reinicia el nivel al mínimo y almacena el récord anterior en la base de datos sin intervención adicional del usuario.
- **Interacción exclusiva:** El bot se utiliza únicamente para tareas de administración y monitorización. La introducción de los números de la secuencia se realiza exclusivamente a través del teclado matricial físico de la consola de juego.

Finalmente, el sistema queda preparado para que el usuario pueda poner a prueba su memoria mediante el juego *Simón Dice IoT*.

Capítulo 6

Resultados y Conclusiones

6.1. Resultados Obtenidos

El proyecto desarrollado cumple satisfactoriamente con los objetivos planteados al inicio del trabajo, demostrando la correcta integración de hardware, software y servicios IoT en un sistema distribuido funcional y escalable.

En primer lugar, el juego de memoria tipo “Simón Dice” funciona de forma estable, permitiendo al usuario interactuar mediante un teclado matricial y recibir feedback inmediato tanto visual (LEDs y pantalla LCD) como sonoro (zumbador). La lógica de validación de la secuencia se ejecuta en tiempo real, detectando errores de forma inmediata y garantizando una experiencia de juego fluida.

La arquitectura distribuida basada en tres nodos ESP32 ha demostrado ser robusta y eficiente. Cada subsistema desempeña su función de manera independiente (entrada, generación de secuencias y feedback), comunicándose mediante MQTT de forma fiable. Esta separación de responsabilidades mejora la mantenibilidad del sistema y facilita futuras ampliaciones.

La integración con Node-RED ha permitido monitorizar el estado del juego en tiempo real, visualizar los resultados de las partidas y gestionar la información de los usuarios de manera intuitiva a través del dashboard. Los datos generados por el sistema se almacenan correctamente en una base de datos MongoDB, permitiendo consultas filtradas por usuario, fecha y nivel alcanzado, así como la generación de estadísticas y gráficas históricas.

Además, el sistema incorpora con éxito un mecanismo de actualización remota de firmware (FOTA), que permite actualizar el software de los dispositivos ESP32 sin necesidad de intervención física. Este mecanismo ha sido probado tanto en el arranque como de forma periódica y mediante disparo manual, demostrando su utilidad para el mantenimiento de sistemas IoT desplegados.

Finalmente, la interfaz de Telegram añade una capa adicional de interacción remota, permitiendo el control y la supervisión del sistema desde dispositivos móviles, lo que refuerza el carácter IoT del proyecto.

6.2. Conclusiones y posibles mejoras

Como conclusión, se puede afirmar que el proyecto ha sido un éxito, ya que se ha conseguido implementar un sistema IoT completo, funcional y bien estructurado, combinando electrónica, comunicaciones, programación concurrente y servicios en la nube. El uso de FreeRTOS ha permitido gestionar múltiples tareas de forma eficiente, y MQTT ha demostrado ser un protocolo adecuado para la comunicación entre nodos distribuidos. No obstante, existen diversas líneas de mejora y ampliación que podrían abordarse en trabajos futuros:

- **Implementar ESP-NOW** para la comunicación local entre nodos, reduciendo la dependencia de la red WiFi y mejorando la latencia.
- **Añadir un sistema de gestión dinámica de usuarios**, con autenticación o perfiles persistentes.
- **Mejorar la interfaz gráfica del dashboard** incorporando más métricas, rankings o comparativas entre jugadores.
- **Integrar una API REST** que permita el acceso externo a los datos almacenados en la base de datos.
- **Incorporar WiFiManager** para facilitar la configuración inicial de la red sin necesidad de recompilar el firmware.
- **Ampliar el juego con nuevos modos** (tiempo límite, multijugador, dificultad adaptativa, etc.).

En conjunto, este proyecto sienta una base sólida para el desarrollo de sistemas IoT distribuidos más complejos y demuestra la aplicabilidad práctica de los conocimientos adquiridos a lo largo del grado.